AI ASSISTED CODING

LAB-12.5

Jadala Varshini

2303A51758

Batch-11

Task Description #1 (Sorting – Merge Sort Implementation) •
Task: Use AI to generate a Python program that implements
the Merge Sort algorithm.
• Instructions: o Prompt AI to create a function
merge_sort(arr) that sorts a list in ascending order.
o Ask AI to include time complexity and space
complexity in the function docstring. o Verify
the generated code with test cases.
• Expected Output: o A functional Python script implementing Merge Sort
with proper documentation.

PROMPT:

#Write a python code to that implements the merge sort algorithm to sort a list of integers in ascending order.Include time complexity and space complexity analysis in the comments and docstrings

CODE and OUTPUT:

lab5.py > merge

```python
#write a python code to that implements the merge sort algorithm to sort a list of integers in ascending order.Include time complex
def merge_sort(arr):
    """Sorts a list of integers in ascending order using the merge sort algorithm.

    Time Complexity: O(n log n) - The merge sort algorithm divides the list into halves log n times and merges them back together i
    Space Complexity: O(n) - The merge sort algorithm requires additional space for the temporary arrays used during the merging pr

    Args:
        arr (list): A list of integers to be sorted.

    Returns:
        list: A new list containing the sorted integers in ascending order.
    """
    if len(arr) <= 1:
        return arr
    mid = len(arr) // 2
    left_half = merge_sort(arr[:mid])
    right_half = merge_sort(arr[mid:])

    return merge(left_half, right_half)
def merge(left, right):
    """Merges two sorted lists into a single sorted list.

    Args:
        left (list): A sorted list of integers.
        right (list): Another sorted list of integers.

    Returns:
        list: A new list containing all integers from both input lists, sorted in ascending order.
    """
```

```
lab5.py > merge
def merge(left, right):
    """
    merged = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i] < right[j]:
            merged.append(left[i])
            i += 1
        else:
            merged.append(right[j])
            j += 1
    merged.extend(left[i:])
    merged.extend(right[j:])
    return merged
# Example usage
if __name__ == "__main__":
    unsorted_list = [38, 27, 43, 3, 9, 82, 10]
    sorted_list = merge_sort(unsorted_list)
    print("Sorted list:", sorted_list)
```

```
PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS

AIAC\lab5.py'
(base) PS C:\Users\thota\OneDrive\Desktop\AIAC>  c:; cd 'c:\Users\thota\OneDrive\Desktop\AIAC'; & 'C:\Users\thota\anaconda3\p
rs\thota\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '54525' '--' 'C:\Users\thota
AIAC\lab5.py'
Sorted list: [3, 9, 10, 27, 38, 43, 82]
```

Task Description #2 (Searching – Binary Search with AI Optimization)

- Task: Use AI to create a binary search function that finds a target element in a sorted list.
- Instructions:
  - o Prompt AI to create a function binary_search(arr, target) returning the index of the target or -1 if not found.
  - o Include docstrings explaining best, average, and worst-case complexities. o Test with various inputs.
- Expected Output: o Python code implementing binary search with AI-generated comments and docstrings **PROMPT:**

#Write a python code to create a function binary_search(arr,target) returning the index of the target or -1 if notfound.o Include docstrings explaining best, average, and worst-case complexities.o Test with various inputs.

CODE AND OUTPUT:

lab5.py > binary_search

```python
#Write a python code to create a function binary_search(arr,target) returning the index of the target or -1 i
def binary_search(arr, target):
    """
    Perform binary search on a sorted array to find the index of the target element.

    Parameters:
    arr (list): A sorted list of elements.
    target: The element to search for.

    Returns:
    int: The index of the target element if found, otherwise -1.

    Complexity:
    Best Case: O(1) - when the target is at the middle of the array.
    Average Case: O(log n) - when the target is found after several iterations.
    Worst Case: O(log n) - when the target is not found or is at the end of the array.
    """
    left, right = 0, len(arr) - 1
    while left <= right:
        mid = left + (right - left) // 2
        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            left = mid + 1
        else:
            right = mid - 1
    return -1
# Test cases
```

```python
    return -1
# Test cases
if __name__ == "__main__":
    arr = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
    print(binary_search(arr, 5))  # Output: 4
    print(binary_search(arr, 1))  # Output: 0
    print(binary_search(arr, 10)) # Output: 9
    print(binary_search(arr, 11)) # Output: -1
    print(binary_search(arr, -1)) # Output: -1
    print(binary_search(arr, 6))  # Output: 5
    print(binary_search(arr, 0))  # Output: -1
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
4
0
(base) PS C:\Users\thota\OneDrive\Desktop\AIAC>  c:; cd 'c:\Users\thota\OneDrive\Desktop\AIAC'; & 'C:\Users\thota\anacon
rs\thota\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '59743' '--' 'c:\Users\t
AIAC\lab5.py' …
(base) PS C:\Users\thota\OneDrive\Desktop\AIAC>  c:; cd 'c:\Users\thota\OneDrive\Desktop\AIAC'; & 'C:\Users\thota\anacon
rs\thota\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '60781' '--' 'c:\Users\t
AIAC\lab5.py'
4
0
9
-1
-1
5
-1
```

Task Description #3: Smart Healthcare Appointment Scheduling System

A healthcare platform maintains appointment records containing appointment ID, patient name, doctor name, appointment time, and consultation fee. The system needs to:

1. Search appointments using appointment ID.
2. Sort appointments based on time or consultation fee.

Student Task

- Use AI to recommend suitable searching and sorting algorithms.
- Justify the selected algorithms.

• Implement the algorithms in Python.

PROMPT:

#write python code to create A healthcare platform maintains appointment records containing appointment ID, patient name, doctor name, appointment time, and consultation fee. The system needs to:1. Search appointments using appointment ID. 2. Sort appointments based on time or consultation fee.Use suitable searching and sorting algorithms

CODE AND OUTPUT:

lab5.py > HealthcarePlatform > __init__

```python
#write  python code to create A healthcare platform maintains appointment records containing appointment ID, patient
class Appointment:
    """A class to represent an appointment in the healthcare platform."""
    def __init__(self, appointment_id, patient_name, doctor_name, appointment_time, consultation_fee):
        """Initialize the appointment with given details."""
        self.appointment_id = appointment_id
        self.patient_name = patient_name
        self.doctor_name = doctor_name
        self.appointment_time = appointment_time
        self.consultation_fee = consultation_fee
class HealthcarePlatform:
    """A class to manage appointments in the healthcare platform."""
    def __init__(self):
        """Initialize the healthcare platform with an empty list of appointments."""
        self.appointments = []
    def add_appointment(self, appointment):
        """Add an appointment to the platform."""
        self.appointments.append(appointment)
    def search_appointment(self, appointment_id):
        """Search for an appointment by its ID using linear search."""
        for appointment in self.appointments:
            if appointment.appointment_id == appointment_id:
                return appointment
        return None
    def sort_appointments_by_time(self):
        """Sort appointments based on appointment time using built-in sorted function."""
        self.appointments.sort(key=lambda x: x.appointment_time)
    def sort_appointments_by_fee(self):
        """Sort appointments based on consultation fee using built-in sorted function."""
```

```python
class HealthcarePlatform:
    def sort_appointments_by_time(self):
        """Sort appointments based on appointment time using built-in sorted function."""
        self.appointments.sort(key=lambda x: x.appointment_time)
    def sort_appointments_by_fee(self):
        """Sort appointments based on consultation fee using built-in sorted function."""
        self.appointments.sort(key=lambda x: x.consultation_fee)
    def display_appointments(self):
        """Display all appointments in the platform."""
        for appointment in self.appointments:
            print(f"ID: {appointment.appointment_id}, Patient: {appointment.patient_name}, Doctor: {appointment.doctor_name}, Time:
# Example usage
if __name__ == "__main__":

    platform = HealthcarePlatform()
    platform.add_appointment(Appointment(1, "John Doe", "Dr. Smith", "2024-07-01 10:00", 100))
    platform.add_appointment(Appointment(2, "Jane Doe", "Dr. Brown", "2024-07-01 11:00", 150))
    platform.add_appointment(Appointment(3, "Alice Johnson", "Dr. Smith", "2024-07-01 09:00", 120))

    print("All Appointments:")
    platform.display_appointments()

    print("\nSearch for Appointment ID 2:")
    appointment = platform.search_appointment(2)
    if appointment:
        print(f"Found: ID: {appointment.appointment_id}, Patient: {appointment.patient_name}, Doctor: {appointment.doctor_name}, Ti
    else:
        print("Appointment not found.")
```

```python
    if appointment:
        print(f"Found: ID: {appointment.appointment_id}, Patient: {appointment.patient_name}, Doctor: {appointment.doctor_name}, Ti
    else:
        print("Appointment not found.")

    print("\nAppointments sorted by time:")
    platform.sort_appointments_by_time()
    platform.display_appointments()

    print("\nAppointments sorted by consultation fee:")
    platform.sort_appointments_by_fee()
    platform.display_appointments()
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
(base) PS C:\Users\thota\OneDrive\Desktop\AIAC>  c:; cd 'c:\Users\thota\OneDrive\Desktop\AIAC'; & 'C:\Users\thota\anaconda3\python.exe' 'c:\Use
rs\thota\.vscode\extensions\ms-python.debugpy-2025.10.0-win32-x64\bundled\libs\debugpy\launcher' '61396' '--' 'c:\Users\thota\OneDrive\Desktop\
AIAC\lab5.py'
All Appointments:
ID: 1, Patient: John Doe, Doctor: Dr. Smith, Time: 2024-07-01 10:00, Fee: 100
ID: 2, Patient: Jane Doe, Doctor: Dr. Brown, Time: 2024-07-01 11:00, Fee: 150
ID: 3, Patient: Alice Johnson, Doctor: Dr. Smith, Time: 2024-07-01 09:00, Fee: 120

Search for Appointment ID 2:
Found: ID: 2, Patient: Jane Doe, Doctor: Dr. Brown, Time: 2024-07-01 11:00, Fee: 150

Appointments sorted by time:
ID: 2, Patient: Jane Doe, Doctor: Dr. Brown, Time: 2024-07-01 11:00, Fee: 150
ID: 3, Patient: Alice Johnson, Doctor: Dr. Smith, Time: 2024-07-01 09:00, Fee: 120

Search for Appointment ID 2:
Found: ID: 2, Patient: Jane Doe, Doctor: Dr. Brown, Time: 2024-07-01 11:00, Fee: 150

Appointments sorted by time:
Found: ID: 2, Patient: Jane Doe, Doctor: Dr. Brown, Time: 2024-07-01 11:00, Fee: 150

Appointments sorted by time:
```

Ln 15, Col 31 Spaces: 4 UTF-8 CRLF {} Python base (3.12

```
else:

PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

Appointments sorted by time:
Appointments sorted by time:
ID: 3, Patient: Alice Johnson, Doctor: Dr. Smith, Time: 2024-07-01 09:00, Fee: 120
ID: 1, Patient: John Doe, Doctor: Dr. Smith, Time: 2024-07-01 10:00, Fee: 100
ID: 2, Patient: Jane Doe, Doctor: Dr. Brown, Time: 2024-07-01 11:00, Fee: 150
ID: 3, Patient: Alice Johnson, Doctor: Dr. Smith, Time: 2024-07-01 09:00, Fee: 120
ID: 1, Patient: John Doe, Doctor: Dr. Smith, Time: 2024-07-01 10:00, Fee: 100
ID: 2, Patient: Jane Doe, Doctor: Dr. Brown, Time: 2024-07-01 11:00, Fee: 150

Appointments sorted by consultation fee:
ID: 1, Patient: John Doe, Doctor: Dr. Smith, Time: 2024-07-01 10:00, Fee: 100
ID: 3, Patient: Alice Johnson, Doctor: Dr. Smith, Time: 2024-07-01 09:00, Fee: 120
ID: 2, Patient: Jane Doe, Doctor: Dr. Brown, Time: 2024-07-01 11:00, Fee: 150
(base) PS C:\Users\thota\OneDrive\Desktop\AIAC>
ID: 3, Patient: Alice Johnson, Doctor: Dr. Smith, Time: 2024-07-01 09:00, Fee: 120
ID: 1, Patient: John Doe, Doctor: Dr. Smith, Time: 2024-07-01 10:00, Fee: 100
ID: 2, Patient: Jane Doe, Doctor: Dr. Brown, Time: 2024-07-01 11:00, Fee: 150

Appointments sorted by consultation fee:
ID: 1, Patient: John Doe, Doctor: Dr. Smith, Time: 2024-07-01 10:00, Fee: 100
ID: 3, Patient: Alice Johnson, Doctor: Dr. Smith, Time: 2024-07-01 09:00, Fee: 120
ID: 1, Patient: John Doe, Doctor: Dr. Smith, Time: 2024-07-01 10:00, Fee: 100
ID: 2, Patient: Jane Doe, Doctor: Dr. Brown, Time: 2024-07-01 11:00, Fee: 150

ID: 1, Patient: John Doe, Doctor: Dr. Smith, Time: 2024-07-01 10:00, Fee: 100
ID: 2, Patient: Jane Doe, Doctor: Dr. Brown, Time: 2024-07-01 11:00, Fee: 150
ID: 2, Patient: Jane Doe, Doctor: Dr. Brown, Time: 2024-07-01 11:00, Fee: 150

Appointments sorted by consultation fee:
ID: 1, Patient: John Doe, Doctor: Dr. Smith, Time: 2024-07-01 10:00, Fee: 100
ID: 3, Patient: Alice Johnson, Doctor: Dr. Smith, Time: 2024-07-01 09:00, Fee: 120
ID: 2, Patient: Jane Doe, Doctor: Dr. Brown, Time: 2024-07-01 11:00, Fee: 150
```

Task Description #4: Railway Ticket Reservation System

Scenario

A railway reservation system stores booking details such as ticket ID, passenger name, train number, seat number, and travel date. The system must:

1. Search tickets using ticket ID.
2. Sort bookings based on travel date or seat number.

Student Task

• Identify efficient algorithms using AI assistance.
• Justify the algorithm choices.
• Implement searching and sorting in Python.

PROMPT:

#write a python program to create a railway reservation sytstem stores booking details such as passenger name,ticket id,train number,seat number,and travel date.you have to search the tickets using ticket id and sort bookings based on travel date or set number. identify the best

algorithmsfor searching and sortingand implement them in your program **CODE AND OUTPUT:**

```python
#write a python program to create a railway reservation sytstem stores booking details such as passenger name,ticket id,train n
class RailwayReservationSystem:
    """A simple railway reservation system to manage booking details."""
    def __init__(self):
        """Initialize the reservation system with an empty list of bookings."""
        self.bookings = []
    def book_ticket(self, passenger_name, ticket_id, train_number, seat_number, travel_date):
        """Book a ticket and store the booking details."""
        booking = {
            'passenger_name': passenger_name,
            'ticket_id': ticket_id,
            'train_number': train_number,
            'seat_number': seat_number,
            'travel_date': travel_date
        }
        self.bookings.append(booking)
    def search_ticket(self, ticket_id):
        """Search for a ticket using its ticket ID."""
        for booking in self.bookings:
            if booking['ticket_id'] == ticket_id:
                return booking
        return None
    def sort_bookings_by_travel_date(self):
        """Sort bookings based on travel date using the built-in sorted function."""
        self.bookings.sort(key=lambda x: x['travel_date'])
    def sort_bookings_by_seat_number(self):
        """Sort bookings based on seat number using the built-in sorted function."""
        self.bookings.sort(key=lambda x: x['seat_number'])
    def display_bookings(self):
        """Display all bookings in the reservation system."""
        for booking in self.bookings:
            print(booking)
```

lab5.py > RailwayReservationSystem > book_ticket

```python
class RailwayReservationSystem:
    def display_bookings(self):
            print(booking)
# Example usage
reservation_system = RailwayReservationSystem()
reservation_system.book_ticket("Alice", "T123", "Train1", "A1", "2024-07-01")
reservation_system.book_ticket("Bob", "T124", "Train2", "B1", "2024-07-02")
reservation_system.book_ticket("Charlie", "T125", "Train1", "A2", "2024-07-01")
print("All Bookings:")
reservation_system.display_bookings()
print("\nSearch for Ticket ID T124:")
ticket = reservation_system.search_ticket("T124")
if ticket:
    print(ticket)
else:
    print("Ticket not found.")
print("\nBookings sorted by Travel Date:")
reservation_system.sort_bookings_by_travel_date()
reservation_system.display_bookings()
print("\nBookings sorted by Seat Number:")
reservation_system.sort_bookings_by_seat_number()
reservation_system.display_bookings()
```

```
reservation_system = RailwayReservationSystem()

PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS

(base) PS C:\Users\thota\OneDrive\Desktop\AIAC>  c:; cd 'c:\Users\thota\OneDrive\Desktop\AIAC'; & 'C:\Users\thota\anaconda3\python.exe' 'c:\Use
rs\thota\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '61625' '--' 'c:\Users\thota\OneDrive\Desktop\
AIAC\lab5.py'
{'passenger_name': 'Alice', 'ticket_id': 'T123', 'train_number': 'Train1', 'seat_number': 'A1', 'travel_date': '2024-07-01'}
{'passenger_name': 'Charlie', 'ticket_id': 'T125', 'train_number': 'Train1', 'seat_number': 'A2', 'travel_date': '2024-07-01'}

Search for Ticket ID T124:
{'passenger_name': 'Bob', 'ticket_id': 'T124', 'train_number': 'Train2', 'seat_number': 'B1', 'travel_date': '2024-07-02'}

Bookings sorted by Travel Date:
{'passenger_name': 'Alice', 'ticket_id': 'T123', 'train_number': 'Train1', 'seat_number': 'A1', 'travel_date': '2024-07-01'}
{'passenger_name': 'Charlie', 'ticket_id': 'T125', 'train_number': 'Train1', 'seat_number': 'A2', 'travel_date': '2024-07-01'}
{'passenger_name': 'Bob', 'ticket_id': 'T124', 'train_number': 'Train2', 'seat_number': 'B1', 'travel_date': '2024-07-02'}

Bookings sorted by Seat Number:
{'passenger_name': 'Alice', 'ticket_id': 'T123', 'train_number': 'Train1', 'seat_number': 'A1', 'travel_date': '2024-07-01'}
{'passenger_name': 'Charlie', 'ticket_id': 'T125', 'train_number': 'Train1', 'seat_number': 'A2', 'travel_date': '2024-07-01'}

Bookings sorted by Seat Number:
{'passenger_name': 'Alice', 'ticket_id': 'T123', 'train_number': 'Train1', 'seat_number': 'A1', 'travel_date': '2024-07-01'}
{'passenger_name': 'Charlie', 'ticket_id': 'T125', 'train_number': 'Train1', 'seat_number': 'A2', 'travel_date': '2024-07-01'}
{'passenger_name': 'Bob', 'ticket_id': 'T124', 'train_number': 'Train2', 'seat_number': 'B1', 'travel_date': '2024-07-02'}
```

Task Description #5: Smart Hostel Room Allocation System A hostel management system stores student room allocation details including student ID, room number, floor, and allocation date. The system needs to:

1. Search allocation details using student ID.
2. Sort records based on room number or allocation date.

Student Task

- Use AI to suggest optimized algorithms.
- Justify the selections.
- Implement the solution in Python.

PROMPT:

#write a python program to create a hostel management system it stores student room allocation details including student id,room number,floor,and allocation date. you have to search allocation details using student id and sort records based on room number or allocation date.you have to sugest optimized algorthms for searching and sorting the records

CODE AND OUTPUT:

lab5.py > ...

```python
#write a python program to create a hostel management system it stores student room allocation details in
class HostelManagementSystem:
    """A simple hostel management system to store student room allocation details."""
    def __init__(self):
        """Initialize the hostel management system."""
        self.allocations = []
    def add_allocation(self, student_id, room_number, floor, allocation_date):
        """Add a new room allocation record."""
        self.allocations.append({
            'student_id': student_id,
            'room_number': room_number,
            'floor': floor,
            'allocation_date': allocation_date
        })
    def search_by_student_id(self, student_id):
        """Search for allocation details using student id."""
        for allocation in self.allocations:
            if allocation['student_id'] == student_id:
                return allocation
        return None
    def sort_by_room_number(self):
        """Sort records based on room number using an optimized sorting algorithm (Timsort)."""
        self.allocations.sort(key=lambda x: x['room_number'])
    def sort_by_allocation_date(self):
        """Sort records based on allocation date using an optimized sorting algorithm (Timsort)."""
        self.allocations.sort(key=lambda x: x['allocation_date'])
    def display_allocations(self):
        """Display all room allocation records."""
        for allocation in self.allocations:
            print(allocation)
# Example usage
hostel_system = HostelManagementSystem()
```

```python
# Example usage
hostel_system = HostelManagementSystem()
hostel_system.add_allocation('S001', '101', '1st Floor', '2024-01-15')
hostel_system.add_allocation('S002', '102', '1st Floor', '2024-01-16')
hostel_system.add_allocation('S003', '201', '2nd Floor', '2024-01-17')
print("Search for student S002:")
print(hostel_system.search_by_student_id('S002'))
print("\nAllocations sorted by room number:")
hostel_system.sort_by_room_number()
hostel_system.display_allocations()
print("\nAllocations sorted by allocation date:")
hostel_system.sort_by_allocation_date()
hostel_system.display_allocations()
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
(base) PS C:\Users\thota\OneDrive\Desktop\AIAC>  c:; cd 'c:\Users\thota\OneDrive\Desktop\AIAC'; & 'C:\Users\thota
rs\thota\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '51977' '--' 'c
AIAC\lab5.py'
{'student_id': 'S002', 'room_number': '102', 'floor': '1st Floor', 'allocation_date': '2024-01-16'}

Allocations sorted by room number:
{'student_id': 'S001', 'room_number': '101', 'floor': '1st Floor', 'allocation_date': '2024-01-15'}
{'student_id': 'S002', 'room_number': '102', 'floor': '1st Floor', 'allocation_date': '2024-01-16'}
{'student_id': 'S003', 'room_number': '201', 'floor': '2nd Floor', 'allocation_date': '2024-01-17'}

Allocations sorted by allocation date:
{'student_id': 'S001', 'room_number': '101', 'floor': '1st Floor', 'allocation_date': '2024-01-15'}
{'student_id': 'S002', 'room_number': '102', 'floor': '1st Floor', 'allocation_date': '2024-01-16'}
{'student_id': 'S003', 'room_number': '201', 'floor': '2nd Floor', 'allocation_date': '2024-01-17'}
(base) PS C:\Users\thota\OneDrive\Desktop\AIAC>

Allocations sorted by allocation date:
{'student_id': 'S001', 'room_number': '101', 'floor': '1st Floor', 'allocation_date': '2024-01-15'}
{'student_id': 'S002', 'room_number': '102', 'floor': '1st Floor', 'allocation_date': '2024-01-16'}
{'student_id': 'S003', 'room_number': '201', 'floor': '2nd Floor', 'allocation_date': '2024-01-17'}
```

Task Description #6: Online Movie Streaming Platform

A streaming service maintains movie records with movie ID, title, genre, rating, and release year. The platform needs to:

1. Search movies by movie ID.
2. Sort movies based on rating or release year.

Student Task

- Recommend searching and sorting algorithms using AI.
- Justify the chosen algorithms.
- Implement Python functions.

PROMPT:

#write a python program to create a streaming service maintains movie records with movie id,title,genre,rating,and release year. you have to search movies by movie id and sort movies based on rating or release year. you have to recommend searching and sorting algorithms for this application and justify your choices with comments and docstrings **CODE AND OUTPUT:**

lab5.py > StreamingService > search_by_id

```python
#write a python program to create a streaming service maintains movie records with movie id,title,genre,rating,and r
class Movie:
    """A class to represent a movie record."""
    def __init__(self, movie_id, title, genre, rating, release_year):
        """Initialize the movie record with its attributes."""
        self.movie_id = movie_id
        self.title = title
        self.genre = genre
        self.rating = rating
        self.release_year = release_year
    def __str__(self):
        """Return a string representation of the movie record."""
        return f"{self.movie_id}: {self.title} ({self.genre}, {self.rating}, {self.release_year})"
class StreamingService:
    """A class to represent a streaming service that maintains movie records."""
    def __init__(self):
        """Initialize the streaming service with an empty list of movies."""
        self.movies = []
    def add_movie(self, movie):
        """Add a movie record to the streaming service."""
        self.movies.append(movie)
    def search_by_id(self, movie_id):
        """Search for a movie by its ID using linear search."""
        for movie in self.movies:
            if movie.movie_id == movie_id:
                return movie
        return None
```

lab5.py > StreamingService > search_by_id

```python
class StreamingService:
    def search_by_id(self, movie_id):
        return None
    def sort_by_rating(self):
        """Sort movies based on rating using the built-in sorted function."""
        return sorted(self.movies, key=lambda x: x.rating, reverse=True)
    def sort_by_release_year(self):
        """Sort movies based on release year using the built-in sorted function."""
        return sorted(self.movies, key=lambda x: x.release_year)
# I have chosen to implement linear search for searching movies by movie ID because it is a simple and straightforward algorith
# For sorting movies based on rating and release year, I have chosen to use the built-in sorted function in Python, which imple
# Example usage:
if __name__ == "__main__":
    service = StreamingService()
    service.add_movie(Movie(1, "Inception", "Sci-Fi", 8.8, 2010))
    service.add_movie(Movie(2, "The Matrix", "Action", 8.7, 1999))
    service.add_movie(Movie(3, "Interstellar", "Sci-Fi", 8.6, 2014))
    print("Search for movie with ID 2:")
    print(service.search_by_id(2))
    print("\nMovies sorted by rating:")
    for movie in service.sort_by_rating():
        print(movie)
    print("\nMovies sorted by release year:")
    for movie in service.sort_by_release_year():
        print(movie)
```

```
Movies sorted by rating:
Search for movie with ID 2:
2: The Matrix (Action, 8.7, 1999)

Movies sorted by rating:
1: Inception (Sci-Fi, 8.8, 2010)
2: The Matrix (Action, 8.7, 1999)
2: The Matrix (Action, 8.7, 1999)
3: Interstellar (Sci-Fi, 8.6, 2014)

Movies sorted by release year:
2: The Matrix (Action, 8.7, 1999)
2: The Matrix (Action, 8.7, 1999)
3: Interstellar (Sci-Fi, 8.6, 2014)

2: The Matrix (Action, 8.7, 1999)
3: Interstellar (Sci-Fi, 8.6, 2014)
2: The Matrix (Action, 8.7, 1999)
3: Interstellar (Sci-Fi, 8.6, 2014)

Movies sorted by release year:
2: The Matrix (Action, 8.7, 1999)
1: Inception (Sci-Fi, 8.8, 2010)
3: Interstellar (Sci-Fi, 8.6, 2014)
```

Task Description #7: Smart Agriculture Crop Monitoring System An agriculture monitoring system stores crop data with crop ID, crop name, soil moisture level, temperature, and yield estimate. Farmers need to:

1. Search crop details using crop ID.
2. Sort crops based on moisture level or yield estimate.

Student Task

- Use AI-assisted reasoning to select algorithms.
- Justify algorithm suitability.
- Implement searching and sorting in Python.

PROMPT:

#write a python program to create an agriculture monitoring system it stores crop data with crop id,crop name,soil moisture level,temperature,and yeild estimate. you have to search crop details using crop id and sort crops based on moisture level or yield estimate .you have to select reasoning algorithm and jusity alorithm and implement searching and sorting **CODE AND OUTPUT:**

```python
#write a python program to create an agriculture monitoring system it stores crop data with crop id,crop name,soil moisture level,t
class Crop:
    """A class to represent a crop in the agriculture monitoring system."""
    def __init__(self, crop_id, crop_name, soil_moisture, temperature, yield_estimate):
        """Initialize the crop with its details."""
        self.crop_id = crop_id
        self.crop_name = crop_name
        self.soil_moisture = soil_moisture
        self.temperature = temperature
        self.yield_estimate = yield_estimate
class AgricultureMonitoringSystem:
    """A class to represent the agriculture monitoring system."""
    def __init__(self):
        """Initialize the monitoring system with an empty list of crops."""
        self.crops = []
    def add_crop(self, crop):
        """Add a crop to the monitoring system."""
        self.crops.append(crop)
    def search_crop_by_id(self, crop_id):
        """Search for a crop by its ID using linear search algorithm."""
        for crop in self.crops:
            if crop.crop_id == crop_id:
                return crop
        return None
    def sort_crops_by_moisture(self):
        """Sort crops based on soil moisture level using bubble sort algorithm."""
        n = len(self.crops)
        for i in range(n):
            for j in range(0, n-i-1):
                if self.crops[j].soil_moisture > self.crops[j+1].soil_moisture:
                    self.crops[j], self.crops[j+1] = self.crops[j+1], self.crops[j]
    def sort_crops_by_yield(self):
        """Sort crops based on yield estimate using bubble sort algorithm."""
        n = len(self.crops)
        for i in range(n):
            for j in range(0, n-i-1):
                if self.crops[j].yield_estimate > self.crops[j+1].yield_estimate:
```

```python
class AgricultureMonitoringSystem:
    def sort_crops_by_yield(self):
        for i in range(n):
            for j in range(0, n-i-1):
                if self.crops[j].yield_estimate > self.crops[j+1].yield_estimate:
                    self.crops[j], self.crops[j+1] = self.crops[j+1], self.crops[j]
# Example usage
if __name__ == "__main__":
    system = AgricultureMonitoringSystem()
    system.add_crop(Crop(1, "wheat", 30, 25, 100))
    system.add_crop(Crop(2, "Corn", 40, 28, 150))
    system.add_crop(Crop(3, "Rice", 20, 22, 120))

    print("Search for crop with ID 2:")
    crop = system.search_crop_by_id(2)
    if crop:
        print(f"Crop found: {crop.crop_name}, Soil Moisture: {crop.soil_moisture}, Temperature: {crop.temperature}, Yield Estimate:
    else:
        print("Crop not found.")

    print("\nCrops sorted by soil moisture level:")
    system.sort_crops_by_moisture()
    for crop in system.crops:
        print(f"{crop.crop_name}: Soil Moisture: {crop.soil_moisture}")

    print("\nCrops sorted by yield estimate:")
    system.sort_crops_by_yield()
    for crop in system.crops:
        print(f"{crop.crop_name}: Yield Estimate: {crop.yield_estimate}")
```

```
Crops sorted by soil moisture level:
Rice: Soil Moisture: 20
Wheat: Soil Moisture: 30
Corn: Soil Moisture: 40
Corn: Soil Moisture: 40

Crops sorted by yield estimate:
Wheat: Yield Estimate: 100
Rice: Yield Estimate: 120
Corn: Yield Estimate: 150
(base) PS C:\Users\thota\OneDrive\Desktop\AIAC>


Corn: Soil Moisture: 40

Crops sorted by yield estimate:
Wheat: Yield Estimate: 100
Rice: Yield Estimate: 120
Corn: Soil Moisture: 40

Crops sorted by yield estimate:
Corn: Soil Moisture: 40

Crops sorted by yield estimate:
Wheat: Yield Estimate: 100
Rice: Yield Estimate: 120
```

Task Description #8: Airport Flight Management System An airport system stores flight information including flight ID, airline name, departure time, arrival time, and status. The system must:

1. Search flight details using flight ID.
2. Sort flights based on departure time or arrival time.

Student Task

- Use AI to recommend algorithms.
- Justify the algorithm selection.
- Implement searching and sorting logic in Python **PROMPT:**

#write a python program to create an airport system store flight information including flight id,airline name,departure time,arrival time,and status. you have to search flight details using flight id and sort bsed on departure time or arrival time..you have to recommend algorithms and justify the algorithm and implement searching and sorting algorithms with comments

CODE AND OUTPUT:

ab5.py > Flight > __init__

```python
#write a python program to create an airport system store flight information including flight id,ai
class Flight:
    """A class to represent a flight in the airport system."""
    def __init__(self, flight_id, airline_name, departure_time, arrival_time, status):
        """Initialize the flight with its details."""
        self.flight_id = flight_id
        self.airline_name = airline_name
        self.departure_time = departure_time
        self.arrival_time = arrival_time
        self.status = status
class AirportSystem:
    """A class to represent the airport system that manages flight information."""
    def __init__(self):
        """Initialize the airport system with an empty list of flights."""
        self.flights = []
    def add_flight(self, flight):
        """Add a flight to the airport system."""
        self.flights.append(flight)
    def search_flight(self, flight_id):
        """Search for a flight by its flight ID using linear search algorithm."""
        for flight in self.flights:
            if flight.flight_id == flight_id:
                return flight
        return None
    def sort_flights_by_departure_time(self):
        """Sort flights based on departure time using the built-in sorted function."""
        self.flights.sort(key=lambda x: x.departure_time)
    def sort_flights_by_arrival_time(self):
        """Sort flights based on arrival time using the built-in sorted function."""
        self.flights.sort(key=lambda x: x.arrival_time)
# Justification for algorithms:
```

lab2 lab3.py launch

lab5.py > Flight > __init__

```python
        self.flights.sort(key=lambda x: x.arrival_time)
# Justification for algorithms:
# 1. Linear Search: I have chosen the linear search algorithm for searching flight details by flight ID because it is simp
# 2. Built-in Sort: I have chosen to use the built-in sorted function for sorting flights based on departure and arrival t
# Example usage:

if __name__ == "__main__":
    airport_system = AirportSystem()
    flight1 = Flight("AA101", "American Airlines", "08:00", "10:00", "On Time")
    flight2 = Flight("DL202", "Delta Airlines", "09:00", "11:00", "Delayed")
    flight3 = Flight("UA303", "United Airlines", "07:30", "09:30", "On Time")
    airport_system.add_flight(flight1)
    airport_system.add_flight(flight2)
    airport_system.add_flight(flight3)

    # Search for a flight
    search_result = airport_system.search_flight("DL202")
    if search_result:
        print(f"Flight found: {search_result.airline_name} departing at {search_result.departure_time}")
    else:
        print("Flight not found.")

    # Sort flights by departure time
    airport_system.sort_flights_by_departure_time()
    print("\nFlights sorted by departure time:")
    for flight in airport_system.flights:
        print(f"{flight.flight_id}: {flight.departure_time}")

    # Sort flights by arrival time
    airport_system.sort_flights_by_arrival_time()
    print("\nFlights sorted by arrival time:")
```

OBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

lab2 lab3.py Launch

lab5.py > Flight > __init__

```python

    # Sort flights by departure time
    airport_system.sort_flights_by_departure_time()
    print("\nFlights sorted by departure time:")
    for flight in airport_system.flights:
        print(f"{flight.flight_id}: {flight.departure_time}")

    # Sort flights by arrival time
    airport_system.sort_flights_by_arrival_time()
    print("\nFlights sorted by arrival time:")
    for flight in airport_system.flights:
        print(f"{flight.flight_id}: {flight.arrival_time}")


```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
e' 'c:\Users\thota\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\
ta\OneDrive\Desktop\AIAC\lab5.py'
Flight found: Delta Airlines departing at 09:00

Flights sorted by departure time:
UA303: 07:30
AA101: 08:00
DL202: 09:00

Flights sorted by departure time:
UA303: 07:30
AA101: 08:00
DL202: 09:00
UA303: 07:30
AA101: 08:00
DL202: 09:00
AA101: 08:00
DL202: 09:00
DL202: 09:00
```

```
ta\OneDrive\Desktop\AIAC\lab5.py'
Flight found: Delta Airlines departing at 09:00

Flights sorted by departure time:
UA303: 07:30
AA101: 08:00
DL202: 09:00

Flights sorted by departure time:
UA303: 07:30
AA101: 08:00
DL202: 09:00
UA303: 07:30
AA101: 08:00
DL202: 09:00
AA101: 08:00
DL202: 09:00
DL202: 09:00

Flights sorted by arrival time:
UA303: 09:30
AA101: 10:00
UA303: 09:30
AA101: 10:00
DL202: 11:00
```